Driver Pulse

Design Document

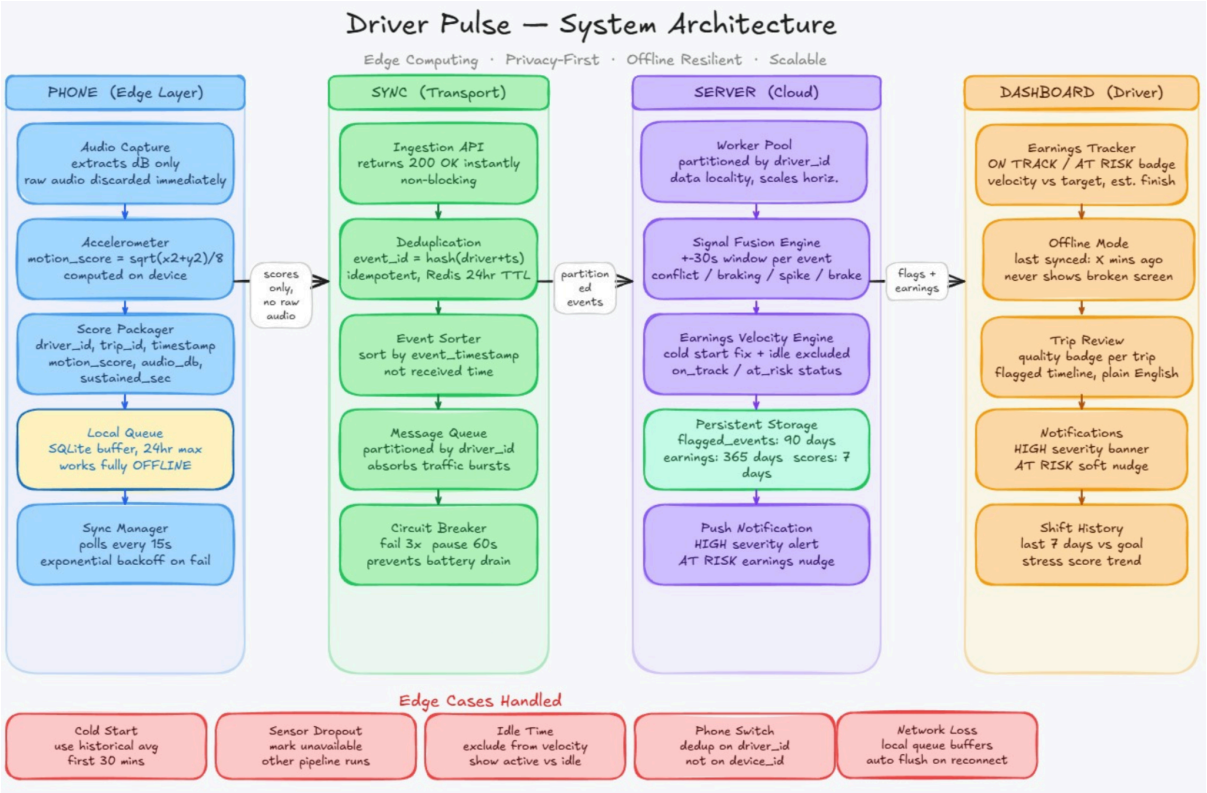
1. Product Vision

Uber drivers finish a shift and see one number: total earnings. They have no idea which moments were stressful, whether a conflict happened in their cab, or whether they were on pace for their goal at any point during the day.

Driver Pulse fixes that. It uses two signals already available on every driver's phone (the accelerometer and the microphone) to give drivers two things after every shift:

- A clear record of stressful or conflict moments during trips, with plain English explanations of what happened and when.
- A live earnings tracker that tells the driver whether they are on pace to hit their daily goal, and projects when they will finish if they keep up their current rate.

The system is built around three principles we committed to from the start. First, privacy: raw audio never leaves the driver's phone. Second, resilience: the system works even with no internet connection. Third, simplicity: the driver should understand everything on screen within two seconds, without reading any instructions.



2. System Overview

The system has four layers. Each layer has a single responsibility and hands off to the next.

```
Phone  ->  Sync  ->  Server  ->  Dashboard
```

- The phone computes scores from sensor data and stores events locally.
- The sync layer receives events, removes duplicates, and queues them for processing.
- The server runs two independent pipelines: one for stress detection, one for earnings.
- The dashboard shows the driver what matters in a glanceable, offline-capable interface.

3. Phone Layer (Edge Computing)

The driver's phone is where all sensitive computation happens. We call this edge computing: instead of sending raw data to a central server, we process it on the device and only send the result. This choice drives three benefits at once: privacy, bandwidth savings, and offline resilience.

3.1 Audio Processing

The microphone reads cabin sound continuously. On the device, we extract the decibel level from the audio signal. The raw audio recording is then discarded immediately. It is never stored anywhere, never transmitted to any server. Only a single number (the dB level) moves forward in the pipeline.

We do not flag every loud moment. A driver playing music or talking to himself would trigger constant false alerts. Instead we apply two conditions before anything gets flagged:

- The dB level must be 88 or above. This threshold corresponds to very loud audio in our dataset, just below argument-level intensity.
- The elevated level must be sustained continuously for at least 30 seconds. A single spike is ignored.

> *Why 88 dB: our dataset showed audio classifications of quiet (52 dB), normal (55-62 dB), conversation (62-68 dB), loud (72-82 dB), very loud (88-92 dB), and argument (94-95 dB). We set the threshold at the start of the very loud range as the minimum level worth monitoring.*

3.2 Motion Processing

The accelerometer reads x, y, and z forces every 30 seconds. We compute a single motion score from these three numbers:

```
motion_score = sqrt(x squared + y squared) / 8.0
```

This gives a value between 0 and 1. The denominator 8.0 was chosen because the most extreme lateral magnitude in our dataset (TRIP002, the reference conflict case) peaked near 8.0. So a score of 1.0 represents the worst event we observed.

We use only x and y (lateral forces) because z measures vertical forces, which mostly reflect road bumps and are not useful for detecting driving behavior.

Why 30 seconds: this matches the sampling interval in the provided dataset. In a production system this would be much finer-grained (accelerometers can sample 50 to 100 times per second), but we work within the data we have.

3.3 Event Packaging and Local Queue

Every 30 seconds, the phone packages the two computed values into a small event and adds it to a local queue stored in SQLite on the device:

driver_id	Identifies which driver this event belongs to
trip_id	Identifies which trip is in progress
timestamp	When the event occurred on the device
motion_score	Lateral acceleration score, 0 to 1
audio_db	Decibel level extracted from microphone
sustained_sec	How long elevated audio has been sustained

The queue keeps up to 24 hours of events. Events older than 24 hours are purged automatically because stale data is not useful for a same-day trip review feature.

3.4 Sync Manager

A background process polls network status every 15 seconds. When a connection is available, it flushes the queue to the server in batches. When offline, it does nothing and retries later. If the server keeps failing, it uses exponential backoff (waiting longer between each retry) to avoid draining the phone's battery.

4. Sync Layer (Transport)

The sync layer's job is pure transport. It receives events from phones, cleans them up, and hands them to the processing queue. It does no computation of its own.

4.1 Ingestion API

The server exposes a POST endpoint that accepts event batches from phones. Critically, it returns 200 OK immediately without waiting to process anything. The phone fires its batch and moves on. This non-blocking design means a phone never hangs waiting for a server response, even under heavy load.

4.2 Deduplication

Because the phone retries failed requests, the same event can arrive at the server more than once. We prevent double-processing by computing a unique ID for each event:

```
event_id = hash(driver_id + timestamp + signal_type)
```

When an event arrives, we check a fast cache for this ID. If we have seen it before, we drop it silently. If it is new, we pass it forward. This means processing the same event twice has the same result as processing it once. The formal term for this property is idempotency.

4.3 Event Sorting

Due to buffering and retry delays, events can arrive at the server in a different order from when they actually occurred. We always sort by event_timestamp (the time the phone recorded the event) rather than by the time the server received it. This ensures our rolling window logic on the server side works correctly.

4.4 Message Queue

Sorted, deduplicated events are dropped into a message queue partitioned by driver_id. Worker processes on the server side consume from their assigned partitions at their own pace. This decouples receiving events from processing them. The server can accept millions of events per second even if processing is slower, because the queue absorbs the burst before it reaches the workers.

4.5 Circuit Breaker

If the server fails to respond three times in a row, the phone stops retrying for 60 seconds before trying again. This prevents a struggling server from being made worse by thousands of phones hammering it simultaneously, and prevents the driver's phone battery from being drained by endless failed requests.

5. Server Layer (Cloud Processing)

The server runs two completely independent pipelines. Signal fusion and earnings velocity share no code and share no data. If one pipeline has a bug and crashes, the other keeps running. This isolation was a deliberate design decision: there is no reason a stress detection failure should break the earnings tracker, or vice versa.

5.1 Worker Pool

Events are consumed by a pool of worker processes. Workers are assigned partitions of driver IDs (worker 1 handles driver IDs 1 to 1000, worker 2 handles 1001 to 2000, and so on). The same driver always routes to the same worker. This is called data locality: the worker already has that driver's context in memory (rolling window history, historical average, current shift earnings, today's goal) so it does not need to fetch it from a database on every single event. Adding more workers linearly increases capacity with no coordination needed between workers.

5.2 Signal Fusion Engine

This pipeline decides whether something bad happened during a trip.

For each incoming event, we convert audio_db to a normalized audio score using the same 0 to 1 scale as motion:

```
audio_score = clip((audio_db - 60) / 40, 0, 1)
```

We then look at a window of plus or minus 30 seconds around each motion event to check whether audio was also elevated at the same time. The 30-second window was chosen because our sampling rate is 30 seconds, giving us one reading on either side. This was validated against TRIP002 in our reference data, where the motion and audio spikes overlap within that window.

Flag rules applied to the combined signals:

Condition	Flag Type	Severity
motion > 0.7 AND audio > 0.7, sustained 30s+	conflict_moment	HIGH
motion > 0.7, audio low	harsh_braking	MEDIUM
audio > 0.7, sustained 30s+, motion low	audio_spike	MEDIUM
motion between 0.4 and 0.7	moderate_brake	LOW

We never flag a single data point in isolation. A pattern across the window is required.

5.3 Earnings Velocity Engine

This pipeline answers one question: is the driver on pace to hit their goal today?

The core calculation is identical to the cricket run rate concept. Current run rate versus required run rate:

```
current_velocity = earnings_so_far / active_hours
required_velocity = target / total_shift_hours
projected = earnings_so_far + current_velocity x
remaining_hours
```

If projected earnings are at or above the target, the driver is ON TRACK. Otherwise AT RISK.

Cold Start Problem

In the first 30 minutes of a shift, the driver has completed very few trips. Computing velocity from this tiny sample gives a wildly unreliable number. We handle this by falling back to the driver's historical average earnings per hour from their profile. After 30 minutes, we switch to the live calculated velocity. On the dashboard, forecasts during the cold start window are labelled as estimated so the driver knows the number is not based on today's data.

Idle Time Problem

A driver waiting 45 minutes for a ride still accumulates elapsed time, which tanks their velocity unfairly. We solve this by computing velocity on active trip time only, excluding idle wait periods. The dashboard shows both active time and idle time separately so the driver can see where their time is going.

5.4 Persistent Storage

flagged_events	Kept for 90 days. Covers trip review and any dispute resolution needs.
earnings_status	Kept for 365 days. Drivers may need earnings records for tax purposes.
raw scores	Purged after 7 days. These are intermediate computation only. Once fusion has run, the raw motion_score and audio_db have served their purpose.

6. Dashboard

The dashboard is what the driver actually sees. Every design decision starts from one word from the problem statement: glanceable. A driver at a red light has one or two seconds. Everything important must be visible in that time.

6.1 Mid-Shift View (Earnings Tracker)

This is the screen a driver sees during an active shift. Four lines of information, readable in two seconds:

Status badge	ON TRACK in green or AT RISK in red. The largest element on the screen.
Progress	Current earnings vs daily goal as a number and progress bar.
Velocity	Current earnings per hour versus required earnings per hour.
Projection	Estimated time the driver will finish if they maintain their current pace.

If there is no network connection, the screen shows the last known data with a small indicator showing how many minutes ago it was last updated. The screen never goes blank or shows an error state.

6.2 Post-Trip View (Trip Review)

After a trip ends, the driver can review what happened. The view has two levels. The top level shows a summary card: trip earnings, duration, distance, and a quality badge (excellent, good, poor, or critical). The driver taps to expand and sees a timeline of any flagged moments, each with a plain English explanation. No raw numbers, no technical labels. The driver does not need to know what m/s squared means.

6.3 Usability Decisions

- Color coding is the primary communication tool. Green means on track. Red means at risk or a critical event. This is universally understood without any tutorial.
- All labels use plain language. ON TRACK not velocity_delta_positive. CONFLICT MOMENT not combined_score_threshold_exceeded.
- Tapping any badge shows a one-line explanation. The app is self-documenting without cluttering the main screen.
- There is no onboarding screen, no tutorial, no manual. A first-time user understands the screen immediately.

7. Edge Cases

We identified five edge cases during design and built explicit handling for each one.

Edge Case	Problem	Fix
Cold Start	In the first 30 minutes, velocity calculation is based on too little data and gives a wildly wrong number.	Fall back to the driver's historical average earnings per hour. Label the forecast as estimated, not calculated. Switch to live velocity after 30 minutes.
Sensor Dropout	Accelerometer stops sending data (battery saver mode, phone off mount).	Detect any gap over 2 minutes. Mark motion data as unavailable. The earnings engine keeps running independently.
Long Idle Wait	Driver waits 45 minutes for a ride. Velocity tanks unfairly because elapsed time grows with no earnings.	Exclude idle time from the velocity calculation. Compute velocity on active trip time only. Show idle time separately on the dashboard.

Phone Switch	Driver changes phones mid-shift. Two device IDs appear for the same driver.	Deduplication is keyed on driver_id and timestamp, not device_id. Both streams merge cleanly into one driver timeline.
Network Loss	Phone enters a tunnel or low coverage area. Events cannot be sent.	Local queue buffers all events on the device. Dashboard shows last known data with a staleness indicator. Queue flushes automatically on reconnect.

8. Privacy Design

Audio privacy was the first constraint we designed around, not an afterthought. The approach has three parts:

- Raw audio is processed on the phone and discarded immediately. It is never stored on the device and never transmitted anywhere.
- Only the computed decibel number is extracted and sent. A number cannot be used to identify a conversation or a person.
- Even the dB number is only used if it crosses a threshold and is sustained for 30 seconds. Single readings below the threshold are never stored.

This design satisfies data minimization: we collect only what is strictly necessary for the feature to work, and nothing more.

9. Scalability

The system was designed to scale to 10 million drivers without architectural changes.

- The message queue absorbs burst traffic. When millions of drivers sync simultaneously (for example, all emerging from a tunnel at the same time), the queue fills up and workers drain it at their own pace. No driver waits and no server crashes.
- Workers are partitioned by driver_id. The same driver always goes to the same worker. Adding more workers increases capacity linearly with no coordination overhead.
- Pipelines are independent. Signal fusion and earnings velocity scale independently based on their own load.
- The phone itself does the heavy lifting for audio and motion computation. The server only receives pre-computed scores, keeping server-side load minimal per event.

10. What We Cut and Why

We deliberately left several features out of this version. We note them here because cutting scope thoughtfully is as important as deciding what to build.

Edge servers in the sync layer	We considered processing events on nearby edge servers to reduce latency. We rejected this because: (1) our insights are post-trip so sub-second latency does not matter, and (2) fusion logic must stay central so thresholds can be updated for all drivers instantly without re-deploying edge infrastructure.
Fusion logic on the phone	Combining the two signals on the phone would reduce server load. We rejected this because threshold updates would require a phone app update to reach 10 million drivers. Keeping fusion on the server means one config change updates every driver instantly.
Idle time pattern analysis	Showing drivers which days or areas have the most idle time would be valuable. We scoped this out to keep focus on the two core features. It is a natural next addition.
Real-time conflict alerts	Pushing a notification the moment a conflict is detected (during the trip) would be useful. We scoped this out because interrupting a driver who is already in a stressful situation may make things worse. Post-trip review is safer.

11. Assumptions and Limitations

- The 30-second sampling interval is taken from the provided dataset. Production accelerometers sample much faster, which would improve detection granularity.
- The motion normalization cap of 8.0 is based on the maximum observed value in TRIP002. A larger dataset would allow a more statistically grounded cap.
- The 88 dB audio threshold is based on the classification labels in the provided dataset. This should be validated with real cabin audio in different vehicle types.
- Cold start fallback uses the driver's historical average. For new drivers with no history, a city-wide average would be used instead.
- The earnings velocity calculation uses active trip time only. The definition of active vs idle is based on whether a trip is in progress. Periods where the driver is driving to pick up a passenger are currently counted as idle.